

Rebuilding OPTIMIZING COMPILER FOR DART

{ @mraleph | vegorov@google.com }

6 YEARS OF DART

(VM engineer perspective)

{ @mrleph | vegorov@google.com }

Excelsior JET

V8

Dart VM

LuaJIT



Dart VM (2012 - ...)

Dart 2011

«Dart targets a wide range of development scenarios: from a one-person project without much structure to a large-scale project needing formal types in the code to state programmer intent. *To support this wide range of projects, Dart has optional types; this means you can start coding without types and add them later as needed.* We believe Dart will be great for writing large web applications.»

— Dart: A language for structured web programming

```
Dog d = new Cat();  
d.woof();
```

```
Dog d = new Cat();  
d.woof();
```

```
class Cat {  
    noSuchMethod(invocation) {  
        if (invocation.memberName == #woof) {  
            print('I am a 🐱 not a 🐶!');  
        }  
    }  
}
```

dynamic language

JIT

initially
AST based

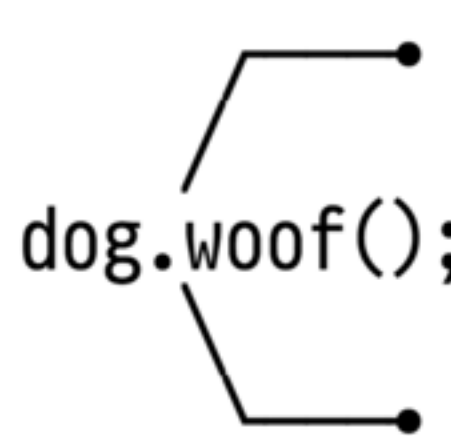
2012

started new **JIT** compiler

inline caching (IC)
+
speculative opts

dog.woof();

- `// call-site specific cache`
`ICData { class ↦ (target, frequency) }`
- `// generic dispatch stub`
`movq rcx, [rbx+0x27]`
`inc dword ptr [rcx+0x87]`
`mov rax, dword ptr [thr+0x20]`
`cmp byte ptr [rax+0x60], 0`
`jnz 0x00000000108a41c8c`
`mov r10, dword ptr [rbx+0x17]`
`mov r13, dword ptr [rbx+0x7]`

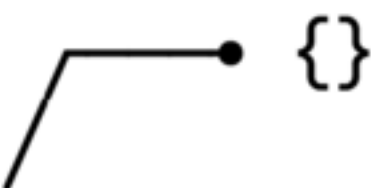


```
// call-site specific cache
ICData { class ↦ (target, frequency) }

dog.woof();

// generic dispatch stub
dispatch(icData, receiver, ...) {
    target = icData.lookup(receiver.classId)
    if (target == null) {
        target = LookupAndCache(icData, receiver)
    }
    invoke target(...)
}
```

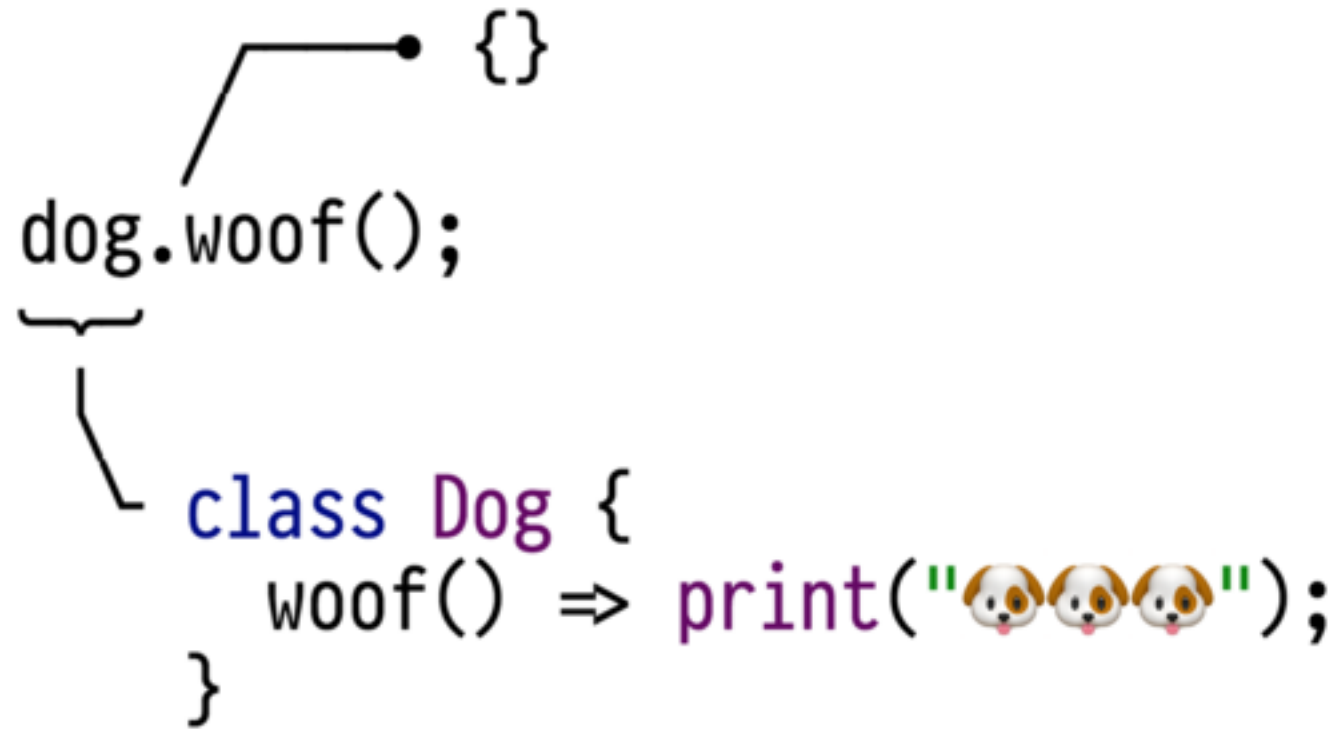
dog.woof();



A diagram illustrating a variable declaration. A line starts from the dot in the code 'dog.woof();', goes diagonally up and to the right, then horizontally to the right, ending at a solid black dot. From this dot, a curly brace '{ }' is positioned to the right.

`dog.woof();`

`class Dog {`
 `woof() => print("🐶🐶🐶");`
`}`



`dog.woof();`

`{`

`LookupAndCache()`

`class Dog {`

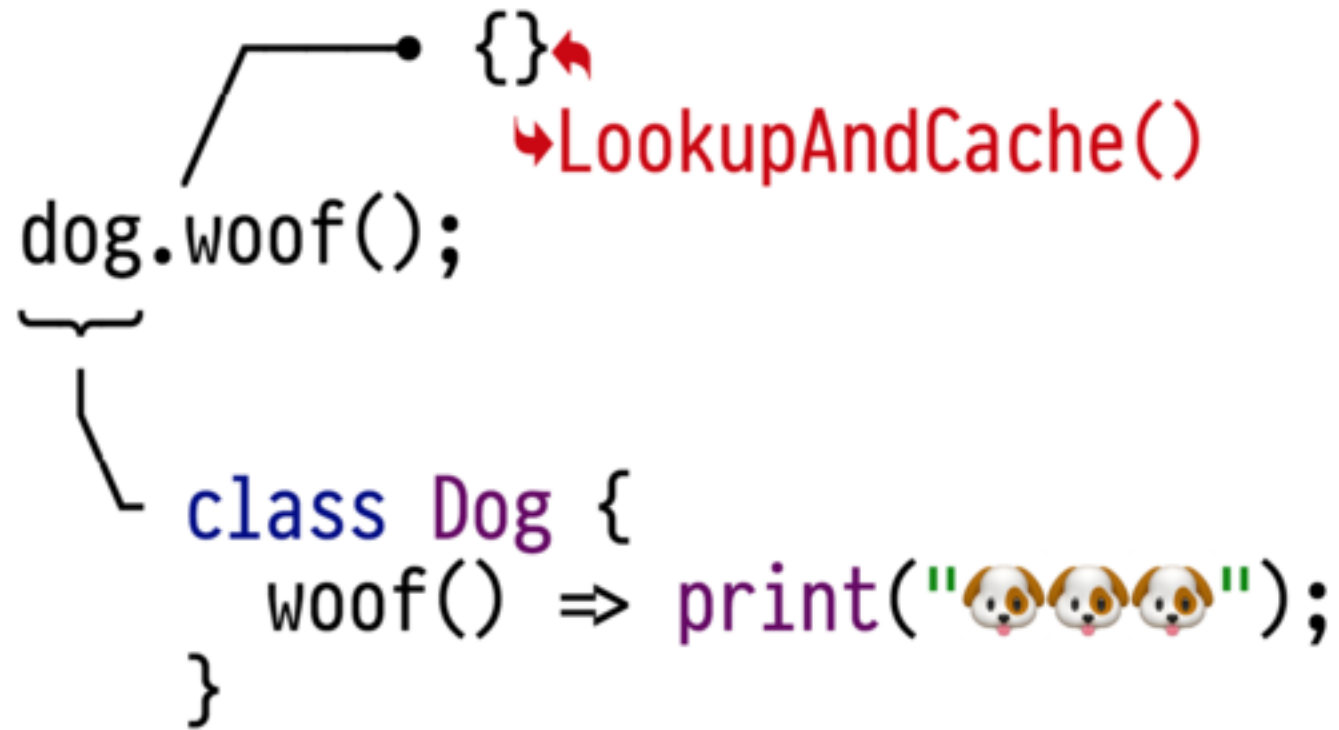
`woof() => print("🐶🐶🐶");`

`}`

`dog.woof();`

`{}`  `LookupAndCache()`

`class Dog {`
 `woof() => print("🐶🐶🐶");`
`}`



`dog.woof();`

$\{ \text{Dog} \mapsto (\text{Dog.woof}, 1) \}$

`class Dog {`
 `woof() => print("🐶🐶🐶");`
`}`

`dog.woof();`

$\{ \text{Dog} \mapsto (\text{Dog.woof}, 2) \}$

`class Dog {`
 `woof() => print("🐶🐶🐶");`
`}`

`dog.woof();`

`{Dog ↦ (Dog.woof, 3)}`

`class Dog {`
 `woof() ⇒ print("🐶🐶🐶");`
`}`

`dog.woof();`

`{Dog ↦ (Dog.woof, 4)}`

`class Dog {`
 `woof() ⇒ print("🐶🐶🐶");`
`}`

`dog.woof();`

$\{ \text{Dog} \mapsto (\text{Dog.woof}, 4) \}$

`class Wolf {`
 `woof() => print("🐺🐺🐺");`
`}`

`dog.woof();`

`{Dog ↦ (Dog.woof, 4)}`
`↳ LookupAndCache() ↗`

`class Wolf {`
 `woof() ⇒ print("🐺🐺🐺");`
`}`

`dog.woof();`

`{Dog ↦ (Dog.woof, 4),
Wolf ↦ (Wolf.woof, 1)}`

`class Wolf {
 woof() ⇒ print("🐺🐺🐺");
}`

type profiling

...
dog.woof();
...
• {Dog $\mapsto$ (Dog.woof, 1000)}

...
dog.wolf();
...

{Dog ↦ (Dog.wolf, 1000)}

// Optimized method IR

...
CheckClass(dog, Dog)
StaticCall(Dog.wolf, dog)
...

... {Dog ↦ (Dog.wolf, 1000)}

▶ dog.wolf();
... // Optimized method IR

deoptimization CheckClass(dog, Dog)
StaticCall(Dog.wolf, dog)
...

The diagram illustrates a deoptimization process. At the top, a black dot is connected by a black line to the text '{Dog ↦ (Dog.wolf, 1000)}'. Below this, the code 'dog.wolf();' is shown with a red arrow pointing to it from the left. To the right of 'dog.wolf();' is the text '// Optimized method IR'. A red line starts from the red arrow, goes down and then right, ending at a red dot next to the text 'CheckClass(dog, Dog)'. Below this is 'StaticCall(Dog.wolf, dog)', and further down is '...'. The word 'deoptimization' is written in red text to the left of the 'CheckClass' line.

classics

Optimizing Dynamically-Typed Object-Oriented Languages With Polymorphic Inline Caches

Urs Hölzle
Craig Chambers
David Ungar[†]

Computer Systems Laboratory, Stanford University, Stanford, CA 94305
{urs,craig,ungar}@self.stanford.edu

Abstract: *Polymorphic inline caches* (PICs) provide a new way to reduce the overhead of polymorphic message sends by extending inline caches to include more than one cached lookup result per call site. For a set of typical object-oriented SELF programs, PICs achieve a median speedup of 11%.

As an important side effect, PICs collect type information by recording all of the receiver types actually used at a given call site. The compiler can exploit this type information to generate better code when *recompiling* a method. An experimental version of such a system achieves a median speedup of 27% for our set of SELF programs, reducing the number of non-inlined message sends by a factor of two.

Implementations of dynamically-typed object-oriented languages have been limited by the paucity of type information available to the compiler. The abundance of the type information provided by PICs suggests a new compilation approach for these languages, *adaptive compilation*. Such compilers may succeed in generating very efficient code for the time-critical parts of a program without incurring distracting compilation pauses.

1. Introduction

Historically, dynamically-typed object-oriented languages have run much slower than statically-typed languages. This disparity in performance stemmed largely from the relatively slow speed and high

surprisingly
general

```
dog.woof(); // Looks like a call, but is actually  
           // a field load and a call.  
└─┬─  
   class Alien {  
     var woof; // A field  
     Alien() : woof = () => print("👁️👁️👁️");  
   }
```

```
//      (...) => dog.woof(...)
var f = dog.woof;
```

method tear-off

dog.woof();

class Cat {
 noSuchMethod(invocation) {
 if (invocation.memberName == #woof) {
 print('I am a 🐱 not a 🐶!');
 }
 }
}

```
dog.woof();  
    {  
        class Cat {  
            noSuchMethod(invocation) {  
                if (invocation.memberName == #woof) {  
                    print('I am a 🐱 not a 🐶!');  
                }  
            }  
        }  
    }
```

`dog.woof();`

`{`

`LookupAndCache()`

`class Cat {`

`noSuchMethod(invocation) {`

`if (invocation.memberName == #woof) {`

`print('I am a 🐱 not a 🐶!');`

`}`

`}`

`}`

`dog.woof();`

`{Cat ↦ ( ?, 1)}`
↳ `LookupAndCache()`

```
class Cat {  
  noSuchMethod(invocation) {  
    if (invocation.memberName == #woof) {  
      print('I am a 🐱 not a 🐶!');  
    }  
  }  
}
```

```
dog.woof();  
    {  
        class Cat {  
            noSuchMethod(invocation) {  
                if (invocation.memberName == #woof) {  
                    print('I am a 🐱 not a 🐶!');  
                }  
            }  
        }  
    }
```

`dog.woof();`

`{`

`LookupAndCache()`

`class Cat {`

`noSuchMethod(invocation) {`

`if (invocation.memberName == #woof) {`

`print('I am a 🐱 not a 🐶!');`

`}`

`}`

`}`

`dog.woof();`

`{`

`LookupAndCache()`

```
class Cat {  
  noSuchMethod(invocation) {  
    if (invocation.memberName == #woof) {  
      print('I am a 🐱 not a 🐶!');  
    }  
  }  
}
```

```
dog.woof();  
    {  
        class Cat {  
            noSuchMethod(invocation) {  
                if (invocation.memberName == #woof) {  
                    print('I am a 🐱 not a 🐶!');  
                }  
            }  
        }  
    }
```

`dog.woof();`

`{`

`LookupAndCache()`

`class Cat {`

`}`

```
dog.woof();  
    {  
    LookupAndCache()  
    }  
  
class Cat {  
    // ↓ injected  
    noSuchMethod:woof() {  
        return noSuchMethod(  
            Invocation(memberName: #woof));  
        }  
    }  
}
```

```
dog.woof();  
  
class Cat {  
    // ↓ injected  
    noSuchMethod:woof() {  
        return noSuchMethod(  
            Invocation(memberName: #woof));  
        }  
    }  
}
```

`dog.woof();`

`{ Cat ↦ (Cat.noSuchMethod:woof, 1) }`

`class Cat {`
 `// ↓ injected`
 `noSuchMethod:woof() {`
 `return noSuchMethod(`
 `Invocation(memberName: #woof));`
 `}`
`}`

suddenly
all optimizations
just work

2012 - 2013

new **JIT** is "done"

non-classical

...
dog.wolf();
...

{Dog ↦ (Dog.wolf, 1000)}

// Optimized method IR

...
CheckClass(dog, Dog)
StaticCall(Dog.wolf, dog)
...

... {Dog ↦ (Dog.wolf, 1000)}
dog.wolf();
...
// Optimized method IR
...
CheckClass(dog, Dog) ← check at use
StaticCall(Dog.wolf, dog)
...

Globally Tracked Field State

```
class DogHolder {  
    var dog = Dog();  
  
    final listOfDogs = List<Dog>(10);  
}
```

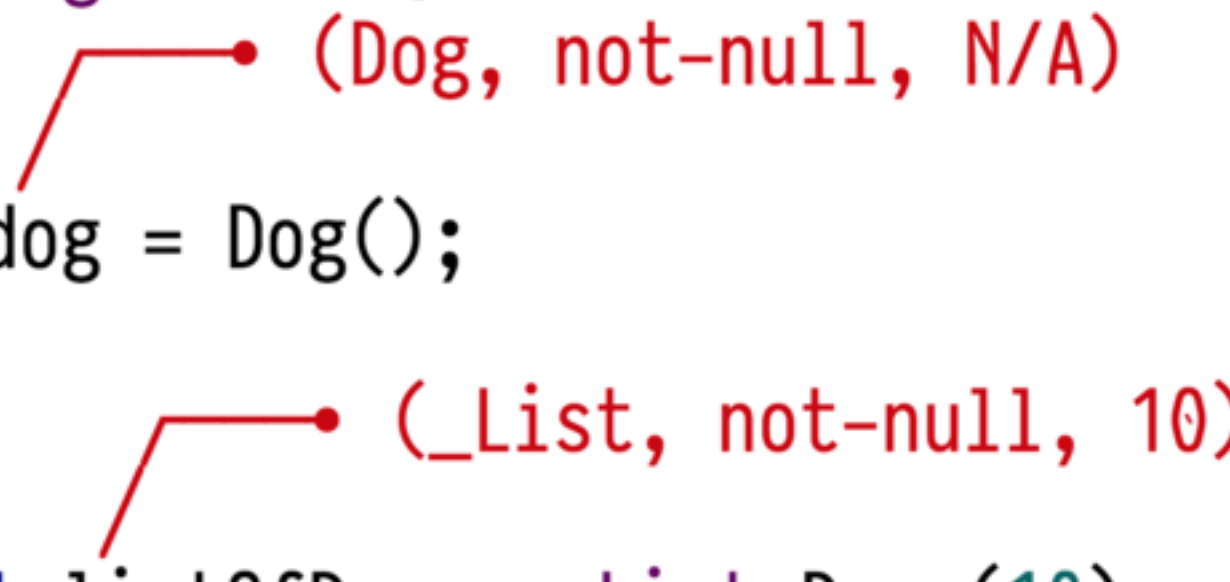
(class, nullability, length-of-array)

(class, nullability, length-of-array)

- Loads can *assume* the state
[Concrete classes are used to eliminate CheckClass, length is is used to eliminate bounds checks.]
- Stores update the state
[If incompatible change: optimized code depending on the assumption is deoptimized]

Globally Tracked Field State

```
class DogHolder {  
    var dog = Dog();  
    final listOfDogs = List<Dog>(10);  
}
```



The diagram illustrates global state annotations for the fields in the `DogHolder` class. A red line connects the `dog` variable to the annotation `(Dog, not-null, N/A)`. Another red line connects the `listOfDogs` variable to the annotation `(_List, not-null, 10)`. The value `10` in the `List` constructor is highlighted in teal.

Optimization shifts checks from uses to definitions

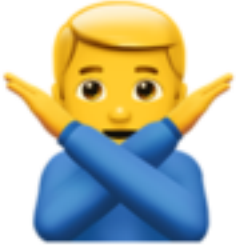
[where they often can be eliminated]

Strange Loop 2013

« Building an Optimizing Compiler for Dart »

2014

tweaking things

« to hand-written assembly» effort

intrinsic rewritten in IL

[planned to move stubs too, still not started on that]

irregexp port

- irregexp is a V8's JITing RE engine
- in V8 it has platform specific backends
- in Dart we just generate IL from RegExp-s

...

« Wait, I thought Dart is
supposed to be a Web
language 🤔 »

VM

dart2js

VM

dart2js

Parallel evolution of Dart-to-JS compilers

- **dartc** [AST based written in Java]
- **frog** [AST based "transpiler" in Dart]
- **leg** [SSA based written in Dart]
- **dart2js** [Actually just renamed leg]

AOT compilation
of Dart to JS
is hard

Object model mismatch

can not completely erase Dart object model in JS
representation only as efficient as JS engine allows it



Open the Sky

Browse files

🔗 master 🏷️ fitness_apk_4 ... demo_apk_22

 **abarth** committed on Oct 23, 2014

1 parent [20cc569](#) commit [ae72930937b1a1522d1919a82ddc21da7d953229](#)

📄 Showing **4,395** changed files with **688,796** additions and **0** deletions.

Unified Split

The diff you're trying to view is too large. We only load the first 3000 changed files.

«What would happen if you
take WebView and remove the
constraint that it has to
render web-pages?»

— Adam Barth at FLOSS Weekly 439

Initially **Sky** started with JS
but later moved to **Dart** *entirely*

[with C++ for lowest *engine* layers]

and renamed



talk from Strange Loop 2017

« Flutter: How we're building a UI framework for tomorrow at Google» by Eric Seidel



is a mobile framework

iOS does not allow JITing

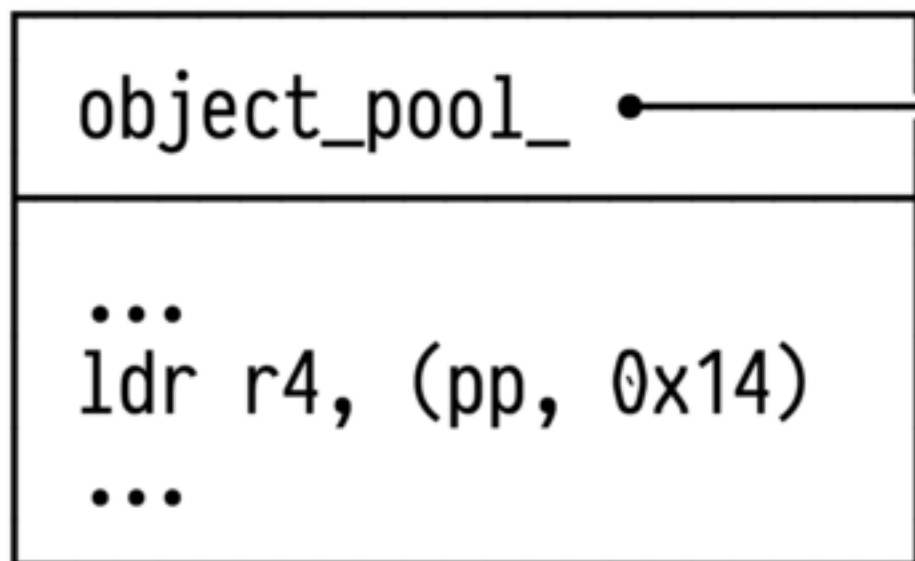
[... unless you are Mobile Safari]

AOT

2015

started AOT prototype

instructions



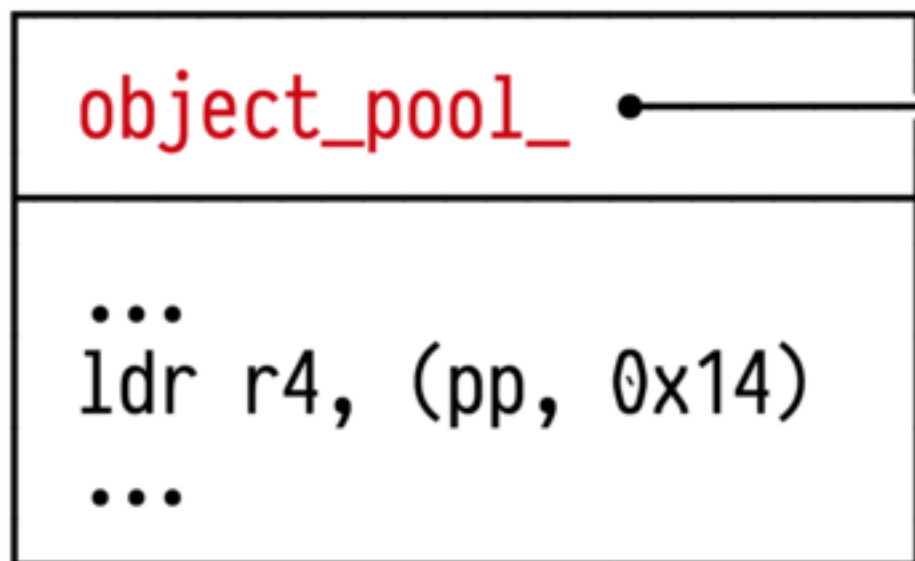
pool



object
in the heap

JIT code
is relocatable

instructions



pool



object
in the heap

had to change calling conventions

[want instructions to be RO and *isolate* independent, while
pools are isolate dependent - so instructions can't contain
pointer to the pool]

« don't want flags & nobody cares about JIT »

[changed calling conventions in a way that impacted JIT performance — in retrospect was a mistake]

then just
serialize it!

already had *snapshots*

[kinda like a heap dump that you can hydrate back]

how to get "JIT" code?

[real JIT does speculative opts!]

disable speculations then
force compile everything

DONE

[prototype took around 2-3 months]

"DONE"

[...]

"DONE"

[...]

AOT compilation
of Dart to JS
is hard

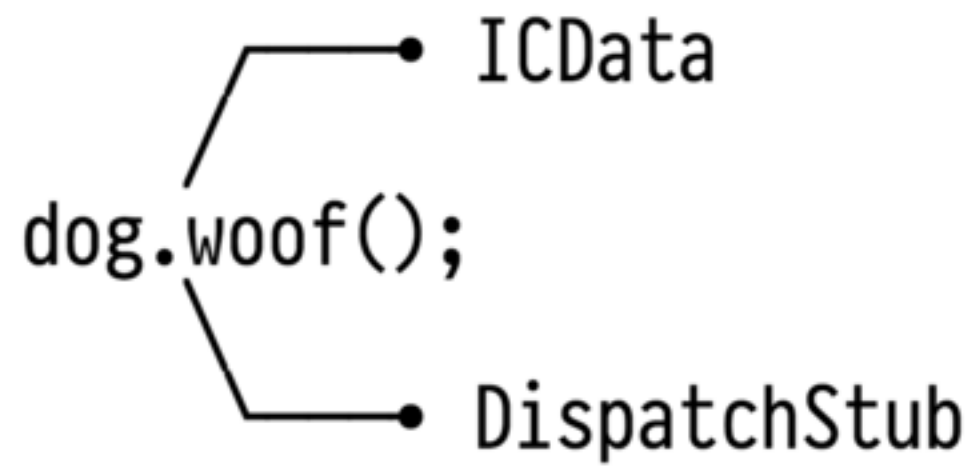
AOT compilation
of Dart to [___]
is nontrivial

need replacement
for **SPECULATIONS**

switchable calls

dog.woof();

- `// call-site specific cache`
`ICData { ... }`
- `// generic dispatch stub`
`dispatch(icData, receiver, ...) { ... }`



code (rx)

pool (rw)

dog.woof();

ICData

can patch these

DispatchStub

code (rx)

pool (rw)

dog.woof();



UnlinkedCall(#woof)

UNLINKED

LinkCall

code (rx)

pool (rw)

dog.woof();



class Dog

MONOMORPHIC

Dog.woof#monomorphicEntry

code (rx)

pool (rw)

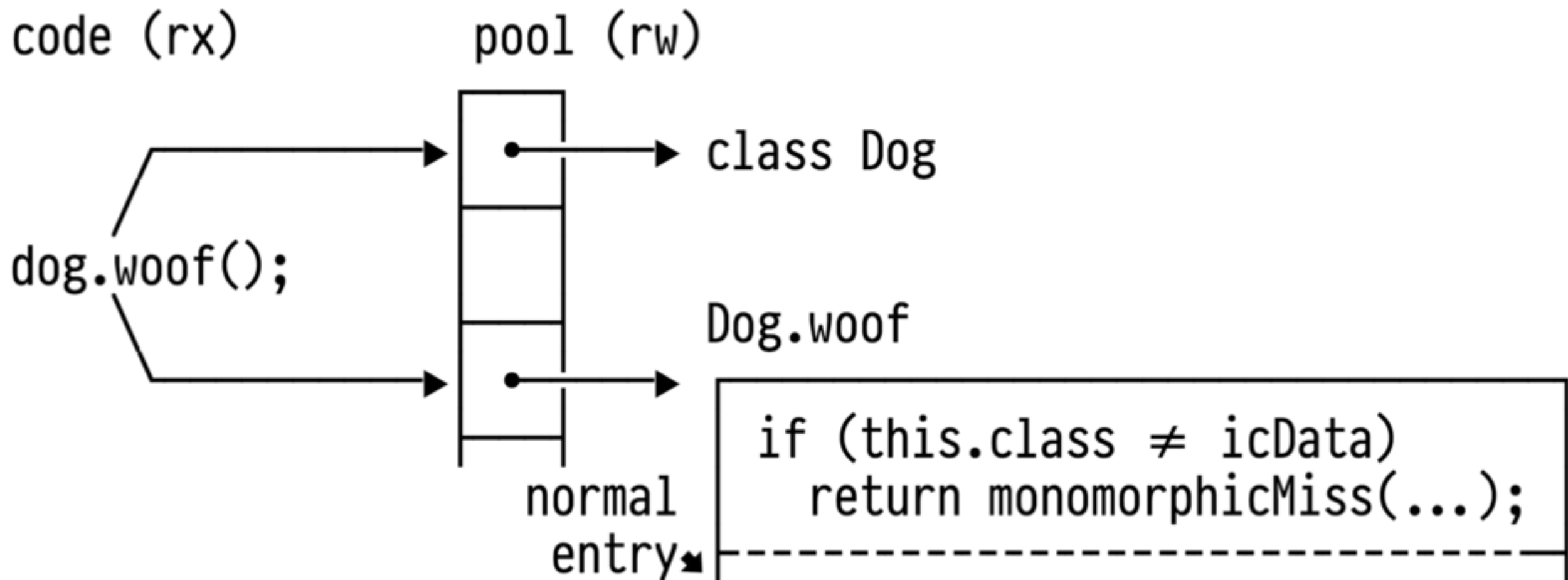
dog.woof();

class Dog

Dog.woof

normal
entry

```
if (this.class != expected)
    return monomorphicMiss(...);
```



code (rx)

pool (rw)

dog.woof();



{ fromCid, toCid, target }

SINGLE TARGET

SingleTargetCallStub

code (rx)

pool (rw)

dog.woof();



ICData { ... }

POLYMORPHIC

ICDispatchStub

code (rx)

pool (rw)

dog.woof();



MegamorphicCache { ... }

MEGAMORPHIC

MegamorphicDispatchStub

inline arithmetic
fast-paths

- operators can be overridden
- `int` is nullable in Dart
- `int` *was* arbitrary precision

```

;; Y ← CheckedSmiOp(+, X, 1)
testl X, 1
jnz slow
movq Y, X
addq Y, 2
jo slow
done:
...
slow: ;; "cold" section of the code
    pushq X
    pushq 1
    movq RBX, [PP+...] ;; megamorphic cache
    callq THR→megamorphic_call_stub
    movq Y, RAX
    jmp done

```

```

graph TD
    Entry(( )) --> Test1[testl X, 1]
    Test1 --> Jnz[jnz slow]
    Jnz --> Slow[slow:]
    Slow --> PushX[pushq X]
    PushX --> Push1[pushq 1]
    Push1 --> MovRBX[movq RBX, [PP+...]]
    MovRBX --> CallStub[callq THR→megamorphic_call_stub]
    CallStub --> MovRAX[movq Y, RAX]
    MovRAX --> JmpDone[jmp done]
    JmpDone --> Done[done:]
    Done --> Ellipsis[...]
    Ellipsis --> Slow

```

```

;; Y ← CheckedSmiOp(+, X, 1)
testl X, 1    ;; what is this if (X & 1 ≠ 0) check?
jnz slow
movq Y, X
addq Y, 2    ;; why are we adding 2?
jo slow
done:
...
slow:        ;; "cold" section of the code
pushq X
pushq 1
movq RBX, [PP+...] ;; megamorphic cache
callq THR→megamorphic_call_stub
movq Y, RAX
jmp done

```

```

graph TD
    Entry(( )) --> Test1[testl X, 1]
    Test1 --> Jnz[jnz slow]
    Jnz --> Slow[slow:]
    Slow --> PushX[pushq X]
    PushX --> Push1[pushq 1]
    Push1 --> MovRBX[movq RBX, [PP+...]]
    MovRBX --> CallStub[callq THR→megamorphic_call_stub]
    CallStub --> MovRAX[movq Y, RAX]
    MovRAX --> JmpDone[jmp done]
    JmpDone --> Done[done:]
    Done --> Entry

```

plus some
closed-world
CHA opts

what about
code size?

code (rx)

pool (rw)

dog.woof();

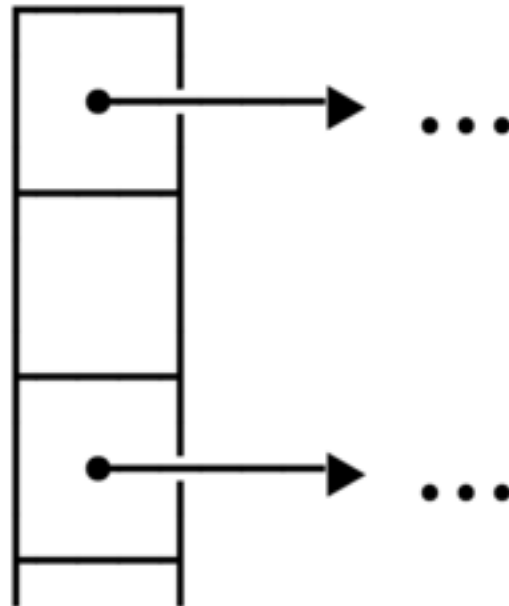
ICData

DispatchStub

code (rx)

pool (rw)

dog.???();



```
void doOne(x) {  
    return x.foo().bar();  
}
```

```
void doTwo(z) {  
    return z.baz().quux();  
}
```

```
void doOne(p0) {  
    return p0.???().???();  
}
```

```
void doTwo(p0) {  
    return p0.???().???();  
}
```

```
void doOne(p0) {  
    return p0.???().???();  
}
```

```
void doTwo(p0) {  
    return p0.???().???();  
}
```



same machine code
and can be shared

byproduct of AOT work

AppJIT snapshots

[warmup the application - then serialize JIT code; massive reduction in warmup time — used by our tools now]

only so much can be done

[remember for example dynamic method injection?]

2016

the wind of changes

(1) dart2js is too slow
for development

Dart Dev Compiler

has been cooking since 2014

Dart Dev Compiler

fast-modular compiles

Dart Dev Compiler

requires stronger type system

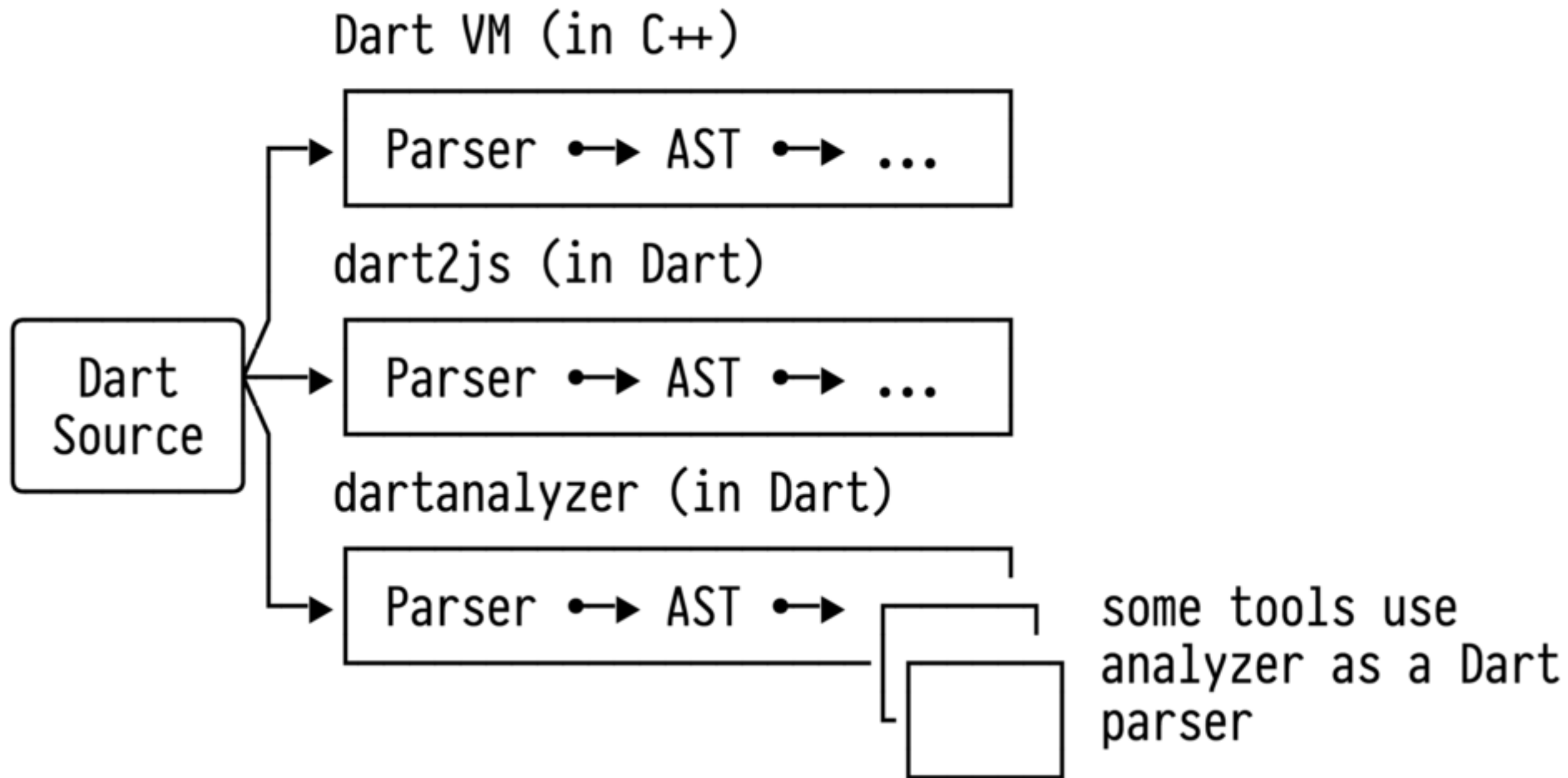
Dog d; // is guaranteed to be a 🐶

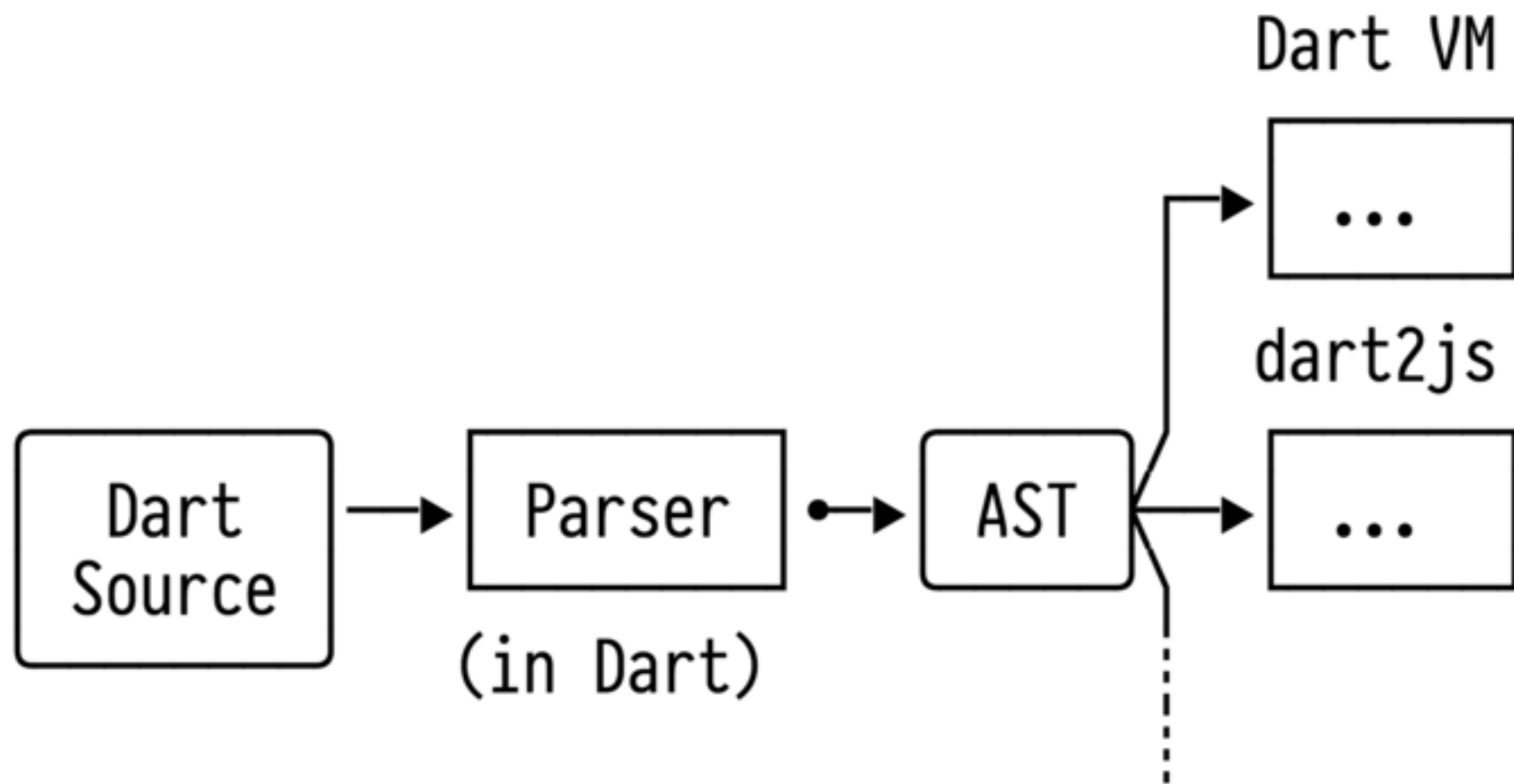
List<Cat> x;
x[0]; // is guaranteed to be a 🐱

talk from Dart Conf 2018

« Evolving Dart: Leaving the
ocean and learning to fly »
by Leaf Petersen

(2) every Dart tool
has its own parser





2016

new type system
& common front-end
migration starts

Aug 2018

Dart 2 shipped

tons of *gnarly* work

[cool stuff too: e.g. started implementing optimization passes in Dart]

is AOT trivial now?

- many important types are *interfaces*, e.g. `List<T>`
- `int` and `double` are still nullable
- `int` became 64-bit wrap around integer
- *soundness* requires runtime type checks

```
// In Dart all generics are covariant  
var strings = ["a", "b"]; // static type List<String>  
strings.add(10);          // compile time error
```

```
// Upcast List<String> to List<Object>  
List<Object> objects = strings; // ok  
objects.add(10);                // runtime error  
                                // requires check somewhere
```

```
// In Dart all generics are covariant
var strings = ["a", "b"]; // static type List<String>
strings.add(10);           // compile time error
```

```
// Upcast List<String> to List<Object>
List<Object> objects = strings; // ok
objects.add(10);               // runtime error
                               // requires check somewhere
```

```
// Also Dart 2 still has dynamic
dynamic x = strings;
x.append("a"); // runtime error
x.add(10);     // runtime error
```

Current approach:

- track type arguments (statically in AOT, dynamically in JIT)
- bypass type checks when runtime type arguments are known to match static type arguments

`List<T>.join(String sep)`

arguments not affected by
covariance – no checks

➤ every call
enters here

`List<T>.join(String sep)`

arguments not affected by
covariance – no checks

➤ every call
enters here
except dynamic ones

```
dynamic x = [...];  
x.join(10);
```

List<T>.join(String sep)

arguments not affected by
covariance – no checks

```
dynamic x = [...];  
x.dyn:join(10);
```

List<T>.join(String sep)

arguments not affected by
covariance – no checks

List<T>.dyn:join(String sep)

Assert(sep is String)
Call List.join •

List<T>.join(String sep)

arguments not affected by
covariance – no checks

dynamic x = [...];
x.join(10);

`List<T>.add(T elem)`

arguments are affected by
covariance – need to check

`List<T>.add(T elem)`

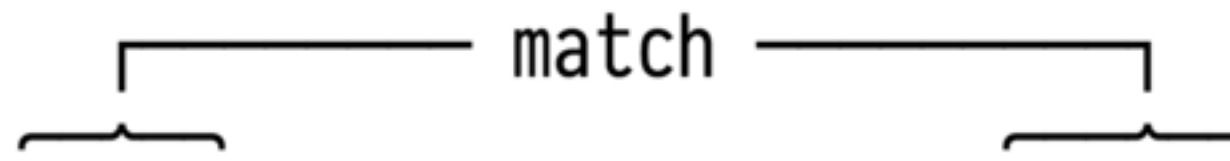
`Assert(elem is T)`

➤ checked entry

➤ unchecked entry

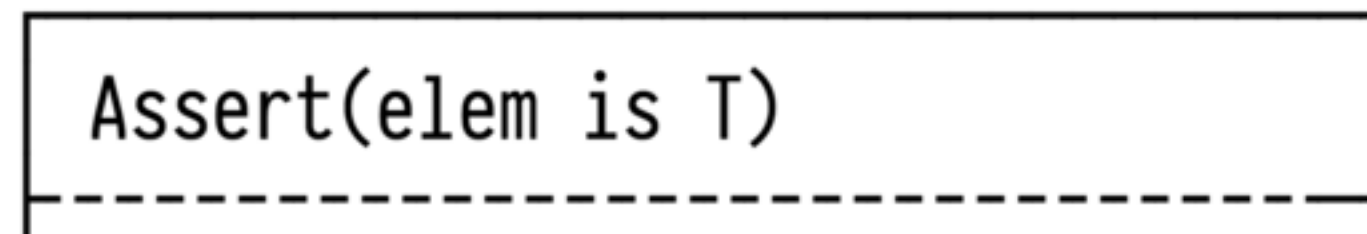
match

```
List<String> strs = new List<String>();  
strs.add(x);
```



```
List<T>.add(T elem)
```

Assert(elem is T)

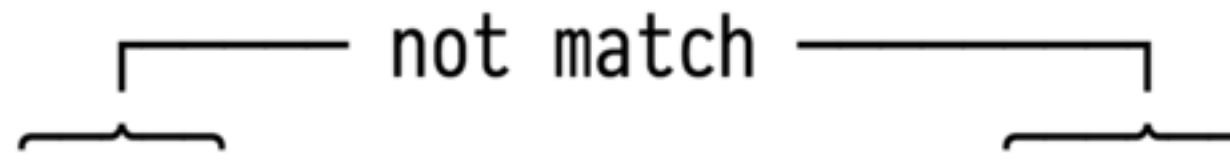


checked entry

unchecked entry

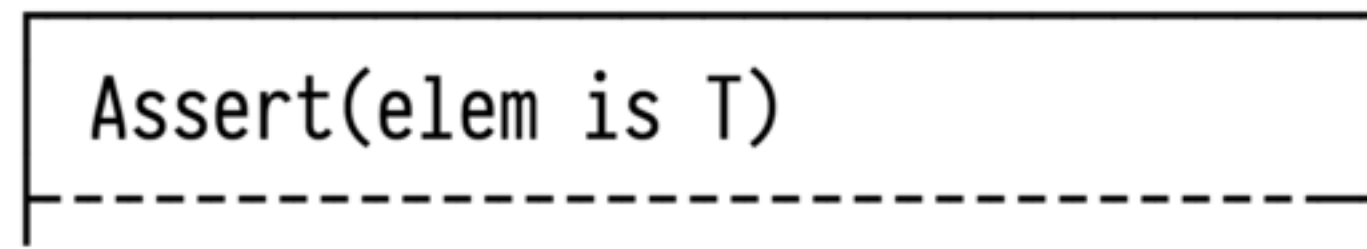
not match

```
List<Object> strs = new List<String>();  
strs.add(x);
```



```
List<T>.add(T elem)
```

Assert(elem is T)



checked entry

unchecked entry

can use static analysis or dynamic profiling
to determine / speculate invariance

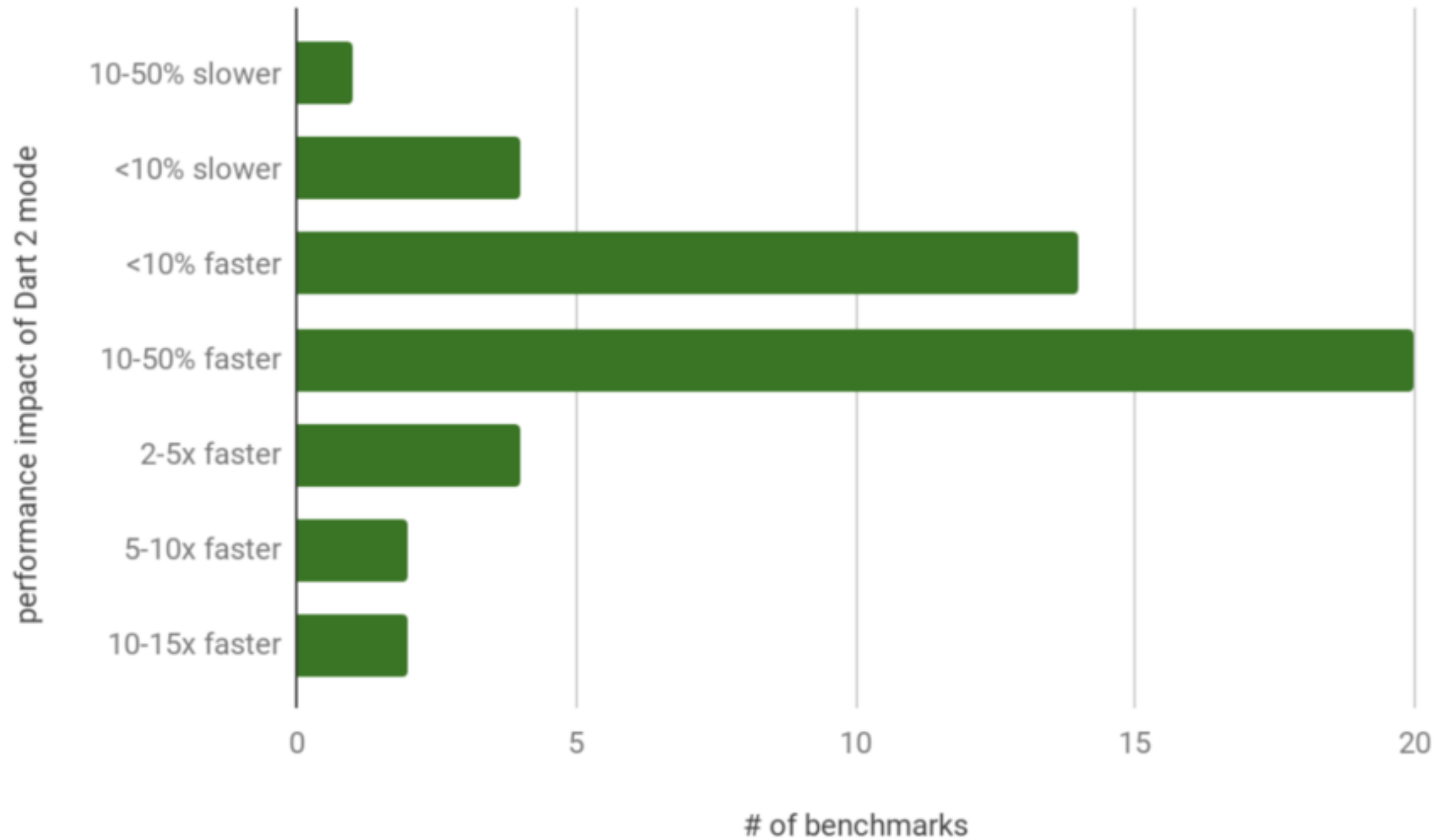
 `List<String> strs = ...;`
`strs.add(x);`

`List<T>.add(T elem)`

`Assert(elem is T)`

checked entry

unchecked entry



architectural changes
take forever

[choose the balance wisely]

parameterize your
architecture

optimizations are more
general than you think

[just need to tweak things a bit]

there are still interesting
problems to solve

[code size for example]

if you have your own
compiler use it more

[don't write assembly by hand]